

Reviewer Pack — Minimal Tropical Rank of Graph Automorphism Groups and Circu...

Entry 5176ee0eb621 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 13:45:52 UTC

Minimal Tropical Rank of Graph Automorphism Groups and Circuit Size Inequality

Entry ID: 5176ee0eb621 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 13:45:52 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every n -vertex graph G , the minimal tropical rank ($\min_tr(G)$) of its associated automorphism group is linearly related to its circuit size $s(C_G)$, such that $\min_tr(G) = O(s(C_G))$ and $\min_tr(G)^2 \geq s(C_G)$.

Rationale (proposer's reasoning):

Tropical geometry offers a framework for studying the complexity of computations in algebraic structures. The minimal tropical rank could provide insights into the structure of graph automorphism groups, which could be leveraged to understand the circuit size required to compute certain functions on graphs.

Taxonomy category: TROPICALGEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dfd777dd1917a49b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -vertex graph G , if the minimal tropical rank ($\min_tr(G)$) is less than or equal to 1.5 times the circuit size $s(C_G)$ AND no seed produces a $\min_tr(G)$ greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Boolean circuit complexity" AND automorphism group" - "min_tr(G)" AND circuit size inequality" - "minimal tropical rank" AND graph automorphism groups AND Boolean circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    augmented_matrix = [row[:] for row in matrix]

    for i in range(rows):
        # Find pivot
        max_row = i
        for r in range(i+1, rows):
            if abs(augmented_matrix[r][i]) > abs(augmented_matrix[max_row][i]):
                max_row = r

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Singular matrix check
        if augmented_matrix[i][i] == 0:
            raise ValueError("Matrix is singular")

        # Eliminate column entries below pivot
        for r in range(i+1, rows):
            factor = -augmented_matrix[r][i] / augmented_matrix[i][i]
            for c in range(cols):
                augmented_matrix[r][c] += factor * augmented_matrix[i][c]

    return augmented_matrix

def min_tropical_rank(matrix):
    try:
        gaussian_eliminated = gaussian_elimination(matrix)
        rank = sum(1 for row in gaussian_eliminated if any(row[j] != 0 for j in range(len(row))))
```

```

        return rank
    except ValueError as e:
        print(f"Error: {e}")
        return None

def circuit_size(graph):
    n = len(graph)
    edges = sum(sum(1 for j in range(i+1, n) if graph[i][j] != 0) for i in range(n))
    return edges * 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = [[random.choice([0, 1]) if i != j else 0 for j in range(n)] for i in range(n)]
        automorphism_group = [] # Placeholder for actual computation
        min_tr_G = min_tropical_rank(automorphism_group)

        if min_tr_G is None:
            return {
                "metric_name": "min_tr(G)",
                "metric_value": None,
                "instances_tested": len(n_values),
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        s_C_G = circuit_size(graph)
        results.append((min_tr_G, s_C_G))

    min_tr_avg = sum(result[0] for result in results) / len(results)
    s_C_avg = sum(result[1] for result in results) / len(results)

    conjecture_holds = all(min_tr <= 1.5 * s_C for min_tr, s_C in results) and max(min_tr for min_tr, s_C in results) <= 1.5 * max(s_C for min_tr, s_C in results)

    return {
        "metric_name": "min_tr(G)",
        "metric_value": min_tr_avg,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

if all(result["conjecture_holds"] for result in results):
    RESULT = "SUPPORTED"
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    RESULT = f"FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}"
else:
    RESULT = "INCONCLUSIVE"

print(f"RESULT: {RESULT} mean={sum(result['metric_value'] for result in results) / len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7206f5e.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7206f5e.py", line 66, in run_trial
    min_tr_G = min_tropical_rank(automorphism_group)
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7206f5e.py", line 46, in min_tropical_rank
    gaussian_eliminated = gaussian_elimination(matrix)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7206f5e.py", line 19, in gaussian_elimination
    rows, cols = len(matrix), len(matrix[0])
                                ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying whether the conjecture's support conditions are met. | next: Review and debug the test code to ensure it can run to completion without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13855
2	preregistration	ollama_remote	glm4:latest	0	0	9294
3	novelty	ollama_remote	glm4:latest	0	0	8338
4	novelty	ollama_remote	glm4:latest	0	0	9014
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19115
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10859
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15885
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11249
9	judge	ollama_remote	glm4:latest	0	0	11479

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109086 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5176ee0eb621.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5176ee0eb621.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5176ee0eb621.tar.gz` (if generated)